

## Apunte 32 — h-index por cluster: cuántos papers son “imperdibles”

Proyecto Froth · aprender Machine Learning construyendo

### 1 El problema (tu propia orden anti-números-mágicos)

En la guía de lectura queríamos marcar los must-reads de cada subtópico. ¿Top 3? No: “top 3” es un número mágico — igual de arbitrario para un cluster de 110 papers de geología que para uno de 9 de tomografía. Pediste *investigar cómo investigar*: un umbral calculado **por corpus y por cluster**.

### 2 La idea robada de la bibliometría: el h-index

El h-index se inventó para medir investigadores, pero la definición funciona sobre CUALQUIER lista de conteos de citas:

Concepto: h-index de un cluster

El mayor  $h$  tal que el cluster tiene  $h$  papers con al menos  $h$  citas cada uno. Ordenas por citas descendentes y buscas el punto donde el rango supera a las citas: justo ahí la evidencia deja de sostenerse a sí misma.

Por qué es elegante: **el propio cluster fija su vara**. Un cluster mega-citado exige cientos de citas para entrar al must-read; uno joven se conforma con decenas. Cero parámetros que tunear.

### 3 Los datos reales de Beauvoir

- Geología/granitos (110 papers)  $\rightarrow h = 35$ : literatura madura, la vara quedó altísima sola.
- Tomografía XCT (9 papers)  $\rightarrow h = 3$ : nicho joven, vara proporcional.
- Informes USGS (mayoría con 0–2 citas)  $\rightarrow$  el h-index degenera ( $h > n/2$ , marcaría “casi todo es imperdible”): ahí entra el **fallback head/tail breaks** — se corta en la media de citas del cluster, que en distribuciones de cola larga separa la cabeza pesada del resto ( $\rightarrow 7$ ).

Ojo / error típico

Citas  $\neq$  calidad: miden atención acumulada, castigan lo reciente (los 59 papers de 2025–2027 con 0 citas son investigación fresca, no mala). El must-read dice “esto es lo canónico que el campo espera que hayas leído”, no “esto es lo único que vale”.

### 4 Dónde vive esta regla (y una lección de 2026-08-03)

Hasta hoy este apunte describía un fallback que **nunca se había programado**: el h-index sí estaba en el código, el corte por la media no. Se implementó al construir el export a Obsidian, en `froth/export_obsidian.py` (`_must_reads` y `_mean_cut`).

## Ojo / error típico

**Un desvío pequeño rompió la regla entera.** Al programarlo, Claude repitió el corte por la media *hasta seis veces* en vez de hacerlo UNA, creyendo que así sería más selectivo. Efecto medido sobre tu corpus de tesis: en el subtema de 340 papers hay un clásico con 4.250 citas que arrastra la media, y al repetir el corte la lista se estrangulaba hasta **1 paper**. Con un solo corte da 36. En total, la versión repetida dejaba con 2 papers o menos a **20 de los 29 subtemas de 30+ papers**; la regla de este apunte no lo hace en ninguno. Lección: cuando un apunte fija un método, el código debe seguirlo *literalmente*. “Lo hago un poco más estricto” no es una mejora, es otro método, y hay que medirlo antes de llamarlo mejor.

Números actuales con la regla correcta: corpus de tesis, 3.867 papers en 104 subtemas → 893 must-reads (mediana 6 por subtema, ninguno se queda corto).

## En una frase

Must-reads sin números mágicos: h-index por cluster (auto-calibrado a la madurez de cada subtópico) con fallback head/tail breaks cuando las citas son demasiado planas para que el h-index discrimine.